

Academic Contribution Registry

Business Requirements

Summary of Problem Statement - Nouns, Verbs

The Academic Contribution Registry is a database system designed to store, organize, and query structured records of research contributions across academic work. The system tracks which researchers have contributed to which publications, and what specific roles they performed in each work. Rather than relying solely on ordered author lists or free-text contribution statements, the system explicitly associates each researcher with a defined contribution role for a given publication.

Each publication may involve multiple researchers, and each researcher may contribute to multiple publications. Additionally, a single researcher may perform multiple roles within the same publication. These relationships must be explicitly recorded and maintained in a structured format.

The system allows users to search for publications by topic, identify researchers by their contribution roles, and analyze collaboration and workload distribution across a research group.

Primary users of the system include researchers, principal investigators (PIs), and administrators. Researchers use the system to find collaborators and review contribution records. PIs use the system to evaluate workload distribution and monitor collaboration patterns. Administrators use the system to generate reports and audit structured contribution data.

The system does not interpret publication content. It only stores structured metadata explicitly entered into the system.

Key Nouns & Verbs

Nouns - Researchers, Roles, Contribution, Publications, Topic, Principal Investigators(PI's), Administrators

Verbs - Store, Organize, Query, Associate, Record, Maintain, Search, Identify, Analyze, Find, Review, Evaluate, Monitor, Generate, Audit

Extracting Classes

- Researchers - name, email_address, ORCID_identifier, institution, department, academic_title
- Publications - title, abstract, publication_year, publication_date, digital_object_identifier (DOI), venue_name, publication_type (journal, conference, preprint), status (published, under review, draft)
- Roles - role_name, role_description, role_category (e.g., writing, analysis, supervision), status (active or inactive)
- Contribution - researcher, publication, role_performed, contribution_notes, date_recorded, verification_status

- Topic - topic_name, topic_description, topic_category
- PublicationTopic - publication, topic, date_associated
- PrincipalInvestigators - name, email_address, institution, department, lab_name
- Administrators - name, email_address, system_role, account_status

Narratives of the System

- A **contribution** represents the association between one **researcher** and one **publication**.
- Each **contribution** must specify exactly one **role** performed by the **researcher** for that **publication**.
- A single **researcher** may have multiple **contributions** for the same **publication** if they performed multiple distinct **roles**.
- Each **role** must be selected from a predefined list of allowable contribution roles (e.g., Conceptualization, Data Curation, Writing, Methodology).
- The system must maintain referential integrity such that no **contribution** can exist without a valid **researcher**, **publication**, and **role**.
- If a **researcher** or **publication** is removed from the system, associated **contributions** must either be restricted or removed according to defined integrity constraints.
The system must store sufficient metadata to uniquely identify a **publication** (e.g., title, year, optional DOI).
- The system must store sufficient metadata to uniquely identify a **researcher** (e.g., name, email, optional ORCID).
- The system must support analytical queries that summarize counts of **contributions** grouped by **researcher**, **role**, or **publication**.

Project 3 Addition — Redis In-Memory Store

Motivation

Projects 1 (SQLite) and 2 (MongoDB) successfully store the full Academic Contribution Registry, but both struggle with one specific query pattern: real-time ranked lookups of the most-viewed publications. In SQLite this requires an expensive ORDER BY COUNT(*) over a view-log table; in MongoDB it requires an aggregation pipeline. Neither is a natural fit for an O(log N) "top-N" lookup that may be requested thousands of times per second on a dashboard.

Functionality moved to Redis

For Project 3 we add a Redis in-memory layer to handle one specific feature: tracking and retrieving the most-viewed publications in the registry. Every time a user views a publication record, we increment a counter in Redis; every time a user opens the "trending" dashboard, we read the top-N ranking directly from Redis.

This Redis layer does NOT replace SQLite or MongoDB. Those remain the system of record for researcher, publication, role, and contribution data. Redis is a high-speed, ranking-optimized overlay for one hot-path query — a classic cache / materialized-view pattern.

Why Redis is the right choice for this functionality

- Atomic increments: ZINCRBY updates a view count in a single round-trip, with no read-modify-write race condition.
- Built-in ranking: ZREVRANGE returns the top-N publications already sorted in $O(\log N + M)$ — no application-side sort needed.
- Random access by rank or score: ZREVRANK and ZSCORE answer "what rank is this publication?" and "how many views does this publication have?" in near-constant time.
- Natural keypace extension: the same pattern supports per-user reading history (mostViewed:<researcherId>) or per-topic rankings (mostViewed:topic:<topicId>) with no code changes.

Data structure

Type: Redis Sorted Set (ZSET)

Key: mostViewed:global

Member: Publication ID (e.g. pub_001)

Score: Integer view count

In-scope for Project 3 (full CRUD on the Redis structure)

- Create — add a publication to the ranking (ZADD)
- Read — top-N, single view count, rank, total count (ZREVRANGE, ZSCORE, ZREVRANK, ZCARD)
- Update — record a view (ZINCRBY) or overwrite a count (ZADD)
- Delete — remove one publication (ZREM) or drop the key (DEL)
- Reset — clear Redis for testing (FLUSHALL)

Challenge Questions

1. Should a **researcher** be uniquely identified by email, ORCID, or a system-generated identifier? What happens if email changes?
2. Are external collaborators (outside the lab or institution) stored as full **researchers**, or should they be marked differently?
3. Should the list of **roles** be strictly controlled by administrators, or can new roles be created dynamically?
4. Should contribution records require verification or approval by a **PI** before becoming official?
5. If a **publication** has an official author order, should that ordering be stored in addition to structured **roles**?
6. Can a **researcher** claim a contribution, or must contributions be assigned by an administrator?
7. Should historical changes to **contributions** be tracked (audit log), or is only the current state required?
8. Are **topics** manually entered tags, or imported from an external metadata source?

AI Disclosure

AI tools (Claude) were used during the preparation of this project for the following tasks: planning the Redis data-structure choice, scaffolding boilerplate Node.js code, and drafting portions of the README and this requirements document. All design decisions, final code, and testing against a live Redis instance were reviewed, edited, and verified by the authors (Neel Vinay and Krishna Raheja).